

Reversing Study

[0x01] : basic 1

Index

- main 함수 찾기
- Quiz

Find the main function

- Requirements
 - WSL 2 / Linux Environment
 - C Programming Skills

Find the main function

```
0 #include <stdio.h>
1
2 int main() {
3     printf("Hello World!\n");
4     return 1;
5 }
```



```
0000000000401136 <main>:
401136:    f3 0f 1e fa                endbr64
40113a:    55                          push    rbp
40113b:    48 89 e5                    mov     rbp, rsp
40113e:    48 8d 05 bf 0e 00 00        lea     rax, [rip+0xebf]
401145:    48 89 c7                    mov     rdi, rax
401148:    e8 f3 fe ff ff            call    401040 <puts@plt>
40114d:    b8 01 00 00 00            mov     eax, 0x1
401152:    5d                          pop     rbp
401153:    c3                          ret
```

Find the main function

.C

```
0 #include <stdio.h>
1
2 int main() {
3     printf("Hello World!\n");
4     return 1;
5 }
```

program (binary)

```
0000000000401136 <main>:
401136: f3 0f 1e fa      endbr64
40113a: 55              push rbp
40113b: 48 89 e5        mov rbp, rsp
40113e: 48 8d 05 bf 0e 00 00 lea rax, [rip+0xebf]
401145: 48 89 c7        mov rdi, rax
401148: e8 f3 fe ff ff  call 401040 <puts@plt>
40114d: b8 01 00 00 00  mov eax, 0x1
401152: 5d              pop rbp
401153: c3              ret
```

process

```
RAX 0x401136 (main) ← endbr64
RBX 0x7fffffffdc68 → 0x7fffffffdf24 ← '/home/littledev0617/Study/Assembly/A'
RCX 0x403e00 (__do_global_dtors_aux_fini_array_entry) → 0x401100 (__do_global_dtors_aux) ←
RDX 0x7fffffffdc78 → 0x7fffffffdf49 ← 'USER=littledev0617'
RDI 1
RSI 0x7fffffffdc68 → 0x7fffffffdf24 ← '/home/littledev0617/Study/Assembly/A'
R8 0
R9 0x7ffff7fca380 (_dl_fini) ← endbr64
R10 0x7fffffff860 ← 0x800000
R11 0x203
R12 1
R13 0
R14 0x403e00 (__do_global_dtors_aux_fini_array_entry) → 0x401100 (__do_global_dtors_aux) ←
R15 0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2e0 ← 0
RBP 0x7fffffffdb40 → 0x7fffffffdc40 ← 0
RSP 0x7fffffffdb48 → 0x7ffff7c2a1ca (__libc_start_call_main+122) ← mov edi, eax
RIP 0x401136 (main) ← endbr64
```

```
► 0x401136 <main>      endbr64
0x40113a <main+4>      push rbp
0x40113b <main+5>      mov rbp, rsp
0x40113e <main+8>      lea rax, [rip + 0xebf]
0x401145 <main+15>     mov rdi, rax
0x401148 <main+18>     call puts@plt

0x40114d <main+23>     mov eax, 1
0x401152 <main+28>     pop rbp
0x401153 <main+29>     ret

0x401154 <_fini>      endbr64
0x401158 <_fini+4>     sub rsp, 8
```

```
RBP ⇒ 0x7fffffffdb40 → 0x7fffffffdb40
RAX ⇒ 0x402004 ← 'Hello World!'
RDI ⇒ 0x402004 ← 'Hello World!'
      <puts@plt>
```

```
EAX ⇒ 1
```

Find the main function

오늘의 목표

- 파일 내에 main 함수 찾기

```
0 #include <stdio.h>
1
2 int main() {
3     printf("Hello World!\n");
4     return 1;
5 }
```



```
0000000000401136 <main>:
401136:    f3 0f 1e fa                endbr64
40113a:    55                          push    rbp
40113b:    48 89 e5                    mov     rbp, rsp
40113e:    48 8d 05 bf 0e 00 00        lea     rax, [rip+0xebf]
401145:    48 89 c7                    mov     rdi, rax
401148:    e8 f3 fe ff ff             call    401040 <puts@plt>
40114d:    b8 01 00 00 00             mov     eax, 0x1
401152:    5d                          pop     rbp
401153:    c3                          ret
```

Find the main function

- gcc

compile - linking $\Rightarrow$ ELF

-no-pie

PIE 보호기법 비활성화

```
0 #include <stdio.h>
1
2 int main() {
3     printf("%p\n", main);
4     return 1;
5 }
```

PIE - 0

```
→ Assembly gcc pie.c -o pie
→ Assembly ./pie
0x646438abc149
→ Assembly ./pie
0x5a916cfc8149
→ Assembly ./pie
0x5a7ee9765149
→ Assembly ./pie
0x58a143ece149
→ Assembly ./pie
0x59e0efffb149 → 0|
```

Handwritten notes: "0" and "1" with arrows pointing to the memory addresses in the assembly output.

PIE - X

```
→ Assembly gcc pie.c -no-pie -o pie
→ Assembly ./pie
0x401136
→ Assembly ./pie
0x401136
→ Assembly ./pie
0x401136
→ Assembly ./pie
0x401136
```

Find the main function

- file

→ Assembly gcc A.c -no-pie -o A

→ Assembly file A

A: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
09b0ac88751183542073, for GNU/Linux 3.2.0, not stripped

1. 파일 생성
2. 링크

- xxd

- s

offset

어디까지

- g

몇 바이트씩 묶을지 (기본 2)

- l

얼만큼

```
→ Assembly xxd ./A
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000 .ELF.....
00000010: 0200 3e00 0100 0000 5010 4000 0000 0000 ..>....P.@...
00000020: 4000 0000 0000 0000 3836 0000 0000 0000 @.....86.....
00000030: 0000 0000 4000 3800 0d00 4000 1f00 1e00 ....@.8...@....
00000040: 0600 0000 0400 0000 4000 0000 0000 0000 .....@.....
00000050: 4000 4000 0000 0000 4000 4000 0000 0000 @.@.....@.@...
00000060: d802 0000 0000 0000 d802 0000 0000 0000 .....
00000070: 0800 0000 0000 0000 0300 0000 0400 0000 .....
00000080: 1803 0000 0000 0000 1803 4000 0000 0000 .....@.....
00000090: 1803 4000 0000 0000 1c00 0000 0000 0000 .....@
```

```
→ Assembly xxd -s 0x1050 -g 1 -l 512 ./A
00001050: f3 0f 1e fa 31 ed 49 89 d1 5e 48 89 e2 48 83 e4 ....1.I..^H..H..
00001060: f0 50 54 45 31 c0 31 c9 48 c7 c7 36 11 40 00 ff .PTE1.1.H..6.@..
00001070: 15 63 2f 00 00 f4 66 2e 0f 1f 84 00 00 00 00 00 .c/...f.....
00001080: f3 0f 1e fa c3 66 2e 0f 1f 84 00 00 00 00 00 90 ....f.....
00001090: b8 18 40 40 00 48 3d 18 40 40 00 74 13 b8 00 00 ..@@.H=..@@.t...
000010a0: 00 00 48 85 c0 74 09 bf 18 40 40 00 ff e0 66 90 ..H..t...@@...f.
000010b0: c3 66 66 2e 0f 1f 84 00 00 00 00 00 0f 1f 40 00 .ff.....@.
000010c0: be 18 40 40 00 48 81 ee 18 40 40 00 48 89 f0 48 ..@@.H...@@.H..H
000010d0: c1 ee 3f 48 c1 f8 03 48 01 c6 48 d1 fe 74 11 b8 ..?H...H..H..t..
000010e0: 00 00 00 00 48 85 c0 74 07 bf 18 40 40 00 ff e0 ....H..t...@@...
000010f0: c3 66 66 2e 0f 1f 84 00 00 00 00 00 0f 1f 40 00 .ff.....@.
00001100: f3 0f 1e fa 80 3d 0d 2f 00 00 00 75 13 55 48 89 .....=/...u.UH.
00001110: e5 e8 7a ff ff ff c6 05 fb 2e 00 00 01 5d c3 90 ..z.....]...
00001120: c3 66 66 2e 0f 1f 84 00 00 00 00 00 0f 1f 40 00 .ff.....@.
00001130: f3 0f 1e fa eb 8a f3 0f 1e fa 55 48 89 e5 48 8d .....UH..H..
00001140: 05 bf 0e 00 00 48 89 c7 e8 f3 fe ff ff b8 01 00 .....H.....
00001150: 00 00 5d c3 f3 0f 1e fa 48 83 ec 08 48 83 c4 08 ..].....H...H...
00001160: c3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001170: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

Find the main function

- readelf

- S

Display sections

#1/1
C

→ text + data entry point

```
→ Assembly readelf -S ./A
There are 31 section headers, starting at offset 0x3638:

Section Headers:
[Nr] Name              Type              Address            Offset
     Size              EntSize          Flags  Link  Info  Align
[ 0]                      NULL              0000000000000000    00000000
     0000000000000000  0000000000000000          0      0      0
[ 1] .interp              PROGBITS          00000000000400318  00000318
     000000000000001c  0000000000000000          A      0      0      1
[ 2] .note.gnu.pr[ ... ]  NOTE              00000000000400338  00000338
     0000000000000030  0000000000000000          A      0      0      8
[ 3] .note.gnu.bu[ ... ]  NOTE              00000000000400368  00000368
     0000000000000024  0000000000000000          A      0      0      4
[ 4] .note.ABI-tag        NOTE              0000000000040038c  0000038c
     0000000000000020  0000000000000000          A      0      0      4
[ 5] .gnu.hash             GNU_HASH           000000000004003b0  000003b0
     0000000000000018  0000000000000000          A      0      0      0
```

```
→ Assembly readelf -S ./A | grep text
[15] .text              PROGBITS          00000000000401050  00001050
```

Virtual Address File Offset

Find the main function

- objdump

- S

view disassembly

- M

architecture

- att (AT&T)

- intel

```
→ Assembly objdump -S -M att ./A
```

```
./A:      file format elf64-x86-64
```

```
Disassembly of section .init:
```

```
0000000000401000 <_init>:
401000:      f3 0f 1e fa      endbr64
401004:      48 83 ec 08      sub     $0x8,%rsp
401008:      48 8b 05 d1 2f 00 00 mov     0x2fd1(%rip),%rax
40100f:      48 85 c0      test    %rax,%rax
401012:      74 02      je      401016 <_init+0x16>
401014:      ff d0      call    *%rax
401016:      48 83 c4 08      add     $0x8,%rsp
40101a:      c3      ret
```

```
→ Assembly objdump -S -M intel ./A
```

```
./A:      file format elf64-x86-64
```

```
Disassembly of section .init:
```

```
0000000000401000 <_init>:
401000:      f3 0f 1e fa      endbr64
401004:      48 83 ec 08      sub     rsp,0x8
401008:      48 8b 05 d1 2f 00 00 mov     rax,QWORD PTR [rip+0x2fd1]
40100f:      48 85 c0      test    rax,rax
401012:      74 02      je      401016 <_init+0x16>
401014:      ff d0      call    rax
401016:      48 83 c4 08      add     rsp,0x8
40101a:      c3      ret
```

Find the main function

- objdump

→ 71상 - 2T1

0000000000401136 <main>:

401136:	f3 0f 1e fa	endbr64	
40113a:	55	push	rbp
40113b:	48 89 e5	mov	rbp, rsp
40113e:	48 8d 05 bf 0e 00 00	lea	rax, [rip+0xebf] # 402004 <_IO_stdin_used+0x4>
401145:	48 89 c7	mov	rdi, rax
401148:	e8 f3 fe ff ff	call	401040 <puts@plt>
40114d:	b8 01 00 00 00	mov	eax, 0x1
401152:	5d	pop	rbp
401153:	c3	ret	

Find the main function

- main 함수를 찾아보자

```
→ Assembly readelf -S ./A | grep text  
[15] .text
```

PROGBITS 00000000000401050

00001050

File Offset

.text

```
00000000000401136 <main>:  
401136: f3 0f 1e fa          endbr64  
40113a: 55                  push    rbp  
40113b: 48 89 e5            mov     rbp, rsp  
40113e: 48 8d 05 bf 0e 00 00 lea     rax, [rip+0xebf]  
401145: 48 89 c7            mov     rdi, rax  
401148: e8 f3 fe ff ff      call   401040 <puts@plt>  
40114d: b8 01 00 00 00      mov     eax, 0x1  
401152: 5d                  pop     rbp  
401153: c3                  ret
```

```
→ Assembly xxd -s 0x1050 -g 1 -l 512 ./A  
00001050: f3 0f 1e fa 31 ed 49 89 d1 5e 48 89 e2 48 83 e4 .....1.I..^H..H..  
00001060: f0 50 54 45 31 c0 31 c9 48 c7 c7 36 11 40 00 ff .PTE1.1.H..6. @..  
00001070: 15 63 2f 00 00 f4 66 2e 0f 1f 84 00 00 00 00 00 .c/ ... f.....  
00001080: f3 0f 1e fa c3 66 2e 0f 1f 84 00 00 00 00 00 90 .....f.....  
00001090: b8 18 40 40 00 48 3d 18 40 40 00 74 13 b8 00 00 ..@@.H=.@@.t....  
000010a0: 00 00 48 85 c0 74 09 bf 18 40 40 00 ff e0 66 90 ..H..t... @@ ... f.  
000010b0: c3 66 66 2e 0f 1f 84 00 00 00 00 00 0f 1f 40 00 .ff..... @..  
000010c0: be 18 40 40 00 48 81 ee 18 40 40 00 48 89 f0 48 ..@@.H... @@.H..H  
000010d0: c1 ee 3f 48 c1 f8 03 48 01 c6 48 d1 fe 74 11 b8 ..?H... H..H..t..  
000010e0: 00 00 00 00 48 85 c0 74 07 bf 18 40 40 00 ff e0 ....H..t... @@ ...  
000010f0: c3 66 66 2e 00 00 00 00 00 00 00 0f 1f 40 00 .ff..... @..  
00001100: f3 0f 1e fa 00 00 00 00 00 00 75 13 55 48 89 .....=./... u.UH..  
00001110: e5 e8 7a ff 00 00 00 00 00 00 01 5d c3 90 ..z.....]..  
00001120: c3 66 66 2e 0f 1f 84 00 00 00 00 00 0f 1f 40 00 .ff..... @..  
00001130: f3 0f 1e fa eb 8a f3 0f 1e fa 55 48 89 e5 48 8d .....UH..H..  
00001140: 05 bf 0e 00 00 48 89 c7 e8 f3 fe ff ff b8 01 00 .....H.....  
00001150: 00 00 5d c3 f3 0f 1e fa 48 83 ec 08 48 83 c4 08 ..].....H... H...  
00001160: c3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....  
00001170: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....  
00001180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

Find the main function

- Install pwndbg
 - `~ git clone https://github.com/pwndbg/pwndbg`
 - `~ cd pwndbg && ./setup.sh`

Find the main function

- gdb
 - b(reakpoint)
 - r(un)
 - s(tep)i(nte)
 - n(ext)i(nte)
- x/[N][Op] <address>
 - Op:
 - [bwg]x : hex dump
 - i : disassembly
 - s : string

```
→ Assembly gcc A.c -no-pie -o A
→ Assembly gdb ./A
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
pwndbg: loaded 194 pwndbg commands. Type pwndbg [filter] for a list.
pwndbg: created 11 GDB functions (can be used with print/break). Type help function
Reading symbols from ./A...
(No debugging symbols found in ./A)
———— tip of the day (disable with set show-tips off) ————
Use the telescope command to dereference a given address/pointer multiple times (if
figure its behavior)
pwndbg> r
Starting program: /home/littledev0617/Study/Assembly/A
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0000000000401136 in main ()
```

```
pwndbg> b*main
Breakpoint 1 at 0x401136
pwndbg> _
```

1. b*main ⇒ 중단점 설정

2. r ⇒ run

```
RAX 0x401136 (main) ← endbr64
RBX 0x7fffffffde88 → 0x7fffffff128 ← '/home/littledev0617/Study/Assembly/A'
RCX 0x403e00 (__do_global_ctors_aux_fini_array_entry) → 0x401100 (__do_global_ctors_aux) ← endbr64
RDX 0x7fffffffde98 → 0x7fffffff14d ← 'HOSTTYPE=x86_64'
RDI 1
RSI 0x7fffffffde88 → 0x7fffffff128 ← '/home/littledev0617/Study/Assembly/A'
R8 0
R9 0x7ffff7fca380 (_dl_fini) ← endbr64
R10 0x7fffffffda80 ← 0x800000
R11 0x203
R12 1
R13 0
R14 0x403e00 (__do_global_ctors_aux_fini_array_entry) → 0x401100 (__do_global_ctors_aux) ← endbr64
R15 0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2e0 ← 0
RBP 0x7fffffffde00 → 0x7fffffffde60 ← 0
RSP 0x7fffffffdd68 → 0x7ffff7c2a1ca (__libc_start_call_main+122) ← mov edi, eax
RIP 0x401136 (main) ← endbr64
```

```
b> 0x401136 <main>      endbr64
    0x40113a <main+4>    push    rbp
    0x40113b <main+5>    mov     rbp, rsp
    0x40113e <main+8>    lea     rax, [rip + 0xebf]
    0x401145 <main+15>   mov     rdi, rax
    0x401148 <main+18>   call   puts@plt

    0x40114d <main+23>   mov     eax, 1
    0x401152 <main+28>   pop     rbp
    0x401153 <main+29>   ret
```

[DISASM / x86-64 / set emulate on]

```
RBP ⇒ 0x7fffffffdd60 → 0x7fffffffde00 → 0x7fffffffde60 ←
RAX ⇒ 0x402004 ← 'Hello World!'
RDI ⇒ 0x402004 ← 'Hello World!'
<puts@plt>
EAX ⇒ 1
```

```
0x401154 <_fini>      endbr64
0x401158 <_fini+4>    sub     rsp, 8
```

[STACK]

```
00:0000 | rsp 0x7fffffffdd68 → 0x7ffff7c2a1ca (__libc_start_call_main+122) ← mov edi, eax
01:0008 | -090 0x7fffffffdd70 → 0x7fffffffddb0 → 0x403e00 (__do_global_ctors_aux_fini_array_entry) → 0x401100 (__do_global_ctors_aux) ← endbr64
02:0010 | -088 0x7fffffffdd78 → 0x7fffffffde88 → 0x7fffffff128 ← '/home/littledev0617/Study/Assembly/A'
03:0018 | -080 0x7fffffffdd80 ← 0x100400040 /* '@' */
04:0020 | -078 0x7fffffffdd88 → 0x401136 (main) ← endbr64
05:0028 | -070 0x7fffffffdd90 → 0x7fffffffde88 → 0x7fffffff128 ← '/home/littledev0617/Study/Assembly/A'
06:0030 | -068 0x7fffffffdd98 ← 0x9cbc95b1424af232
07:0038 | -060 0x7fffffffdda0 ← 1
```

[BACKTRACE]

```
► 0 0x401136 main
  1 0x7ffff7c2a1ca __libc_start_call_main+122
  2 0x7ffff7c2a28b __libc_start_main+139
  3 0x401075 _start+37
```


Quiz

1. 아무 C 코드 컴파일하여 생성된 ELF 바이너리 내 main 함수 objdump, xxd 로 찾아서 스크린샷

- compile option: 64bit / dynamic linking / not stripped / no PIE

- objdump
main 함수 Virtual Address 확인

- readelf
.text section 주소 및 File Offset 확인

⇒ .text section 으로부터의 main 함수 주소 offset 확인

- xxd
.text section File Offset + main 함수 offset ⇒ main 함수 File Offset!!

위 과정 간단하게 캡처해서 디스코드 DM 보내주시면 됩니다

Quiz

2. 다음 코드 실행 후 생성되는 A2 실행한 결과 확인

⇒ 왜 이런 결과가 나왔을까?

한 번 생각해보기! 필수 X, 질문 or 답 디스코드 DM

```
0 data = bytearray(open('./A', 'rb').read())
1
2 offset = int.from_bytes(data[0x1141:0x1141+4], 'little')
3 data[0x1141:0x1141+4] = (offset + 6).to_bytes(4, 'little')
4
5 open('./A2', 'wb').write(data)
```

```
→ basic1 vim a.py
→ basic1 python3 a.py
→ basic1 chmod +x ./A2
→ basic1 ./A2
World!
```

Handwritten notes in blue ink:

↑ 16
12/A.1
IF

* Python

- bytes

`[n]` $\Rightarrow$ n 번째 값 $\Rightarrow$ `int`

`.decode("utf-8")` $\Rightarrow$ `str`

`list(bytes)` $\Rightarrow$ `List[int]` : 각 원소 x 범위 $0 \leq x \leq 255$ 인 `int` list

`.hex()` $\Rightarrow$ `str`, hex string

`.fromhex("ABCD..")` $\Rightarrow$ `bytes`, hex string $\Rightarrow$ bytes

- int

`.to_bytes(N, '(little|big)')` $\Rightarrow$ `bytes`, N 바이트 little/big Endian

`.from_bytes(bytes, '(little|big)')` $\Rightarrow$ `int`, 해당 바이트를 little/big Endian 해석

- str

`.encode()` $\Rightarrow$ `bytes`